TECHNICAL REPORT

Agentic AI System for Clinical Discharge Summary Generation

Assignment	Dscribe AI Engineer Take-Home Task
Project	Agentic Discharge Summary System (Part 1 + Part 2)
Stack	Python · Streamlit · LangGraph · Groq API · EasyOCR · PyMuPDF
LLM	LLaMA 3.3 70B Versatile via Groq
Date	June 2026

Disclaimer: All patient data used during development is fully synthetic. No real patient records were used or stored. API keys are excluded from all code repositories.

Table of Contents

1. Project Overview & Problem Statement

Clinical discharge summaries are high-stakes documents: incomplete, fabricated, or conflicting information can directly harm patients. This project implements a multi-step agentic AI system that ingests raw, messy source-note PDFs for a patient and produces a structured, clinically safe discharge summary draft — always flagging what is missing or uncertain rather than guessing.

Assignment requirements addressed:

Requirement	Status	Implementation
Real agent loop (plan → re-plan)	✓ Done	LangGraph StateGraph with conditional edges
PDF ingestion	✓ Done	PyMuPDF → raster → EasyOCR pipeline
No fabrication guardrail	✓ Done	Hard prompt rule; missing → 'MISSING' label
Pending / missing data handling	✓ Done	Explicit PENDING labels in all outputs
Medication reconciliation	✓ Done	Diff logic enforced in Extractor prompt
Conflicting information flagging	✓ Done	Conflict detection; never silently resolved
Tool use with agent decision	✓ Done	Mock drug-interaction + escalation tools
Robust failure handling	✓ Done	try/except per node + iteration cap
Iteration cap (hard control)	✓ Done	iteration >= 2 terminates graph
Readable step trace / observability	✓ Done	trace[] list emitted per node
Part 2 — Learning loop	✓ Done	Simulated reviewer + memory injection

2. System Architecture

The system is composed of four logical layers that interact sequentially for each patient record:

Layer	Components	Responsibility
Input Layer	Streamlit uploader PyMuPDF EasyOCR	Accept PDF / image uploads; convert PDF pages to raster images; run OCR to produce raw text.
Agent Layer	LangGraph StateGraph Planner Node Extractor Node Conditional Edge	Orchestrate multi-step reasoning; plan extraction goals; extract & reconcile clinical facts; loop until done or iteration cap reached.
LLM Layer	Groq API LLaMA 3.3 70B Versatile	Power all reasoning tasks: planning, extraction, conflict detection, and medication reconciliation.
Output / UI Layer	Streamlit panel Audit trail expander Sidebar feedback	Render structured discharge summary draft; expose step-by-step trace; collect doctor feedback for Part 2.

Data flow — step by step:

1. User uploads one or more PDFs (or images) via Streamlit.
2. PyMuPDF opens each PDF and renders every page to a pixel-map image.
3. EasyOCR processes each image and returns detected text strings.
4. All text is concatenated into raw_text and passed as the initial AgentState.
5. LangGraph invokes the Planner → Extractor cycle, up to the iteration cap.
6. The final summary and trace are displayed; doctor feedback is saved to session memory.

3. Technology Stack — Detailed Justification

Every component was chosen deliberately. The subsections below explain each choice and why alternatives were rejected.

3.1 LLM: LLaMA 3.3 70B Versatile via Groq

The LLM is the reasoning core. Choosing the right model and provider determines output quality, latency, and cost.

Criterion	LLaMA 3.3 70B (Groq)	GPT-4o (OpenAI)	Gemini 1.5 (Google)	Mixtral 8x7B
Cost	Free tier	$5–15/M tokens	Free tier limited	Free but weaker
Latency	< 1s (LPU)	2–8s typical	2–5s typical	~1–2s
Clinical reasoning	Strong (70B)	Excellent	Good	Moderate
Instruction follow	Excellent	Excellent	Good	Good
Structured output	Reliable	Excellent	Good	Variable
Open-source	Yes (Meta)	No	No	Yes
API setup	Simple, free	Paid only	GCP required	Multiple providers

Why LLaMA 3.3 70B:

- 70B parameters gives near-GPT-4 reasoning without GPT-4 cost — critical for clinical extraction depth.
- Groq's LPU hardware delivers sub-second inference; essential when every agent node calls the LLM independently.
- Open-weight model means auditable behaviour — important in a clinical context requiring explainability.
- Free Groq tier is sufficient for prototyping; production would use paid Groq or self-hosted deployment.

Why NOT the alternatives:

- **GPT-4o:** Excellent quality but expensive per token, closed-source, and requires a paid API key — inappropriate for a free assignment demo.
- **Gemini 1.5 Pro:** Strong model, but Google Cloud setup adds friction; instruction-following on structured clinical extraction is marginally weaker.
- **Mixtral 8x7B:** Faster and cheaper but at 46.7B effective parameters (MoE) it visibly hallucinates medication names — directly violating the no-fabrication guardrail.
- **LLaMA 3.1 8B:** Too small: tends to fabricate clinical facts and cannot reliably reconcile medication lists.

3.2 Orchestration: LangGraph

LangGraph provides the agent loop — state management, node wiring, conditional routing, and iteration control.

Framework	Agent loop	State mgmt	Conditional edges	Learning curve	Production
LangGraph	✓	TypedDict	Native ✓	Medium	Yes
CrewAI	✓	Role-based	Limited	Low	Beta
AutoGen	✓	Conversation	Limited	Medium	Beta
Raw loops	Manual	Manual	Manual	High	Fragile

Framework	Agent loop	State mgmt	Conditional edges	Learning curve	Production
LangChain	Chain only	Partial	No	Medium	Yes

LangGraph was chosen because it natively supports stateful, cyclical graphs with conditional branching. CrewAI's role abstraction hides the loop logic, reducing trace readability. A raw Python loop lacks LangGraph's state propagation guarantees.

3.3 PDF Ingestion: PyMuPDF + EasyOCR

Clinical notes arrive as PDFs that may be scanned images with no text layer. A two-stage pipeline is required: render pages to images, then OCR.

Stage	Chosen tool	Alternative	Why chosen
PDF → image	PyMuPDF (fitz)	pdf2image / poppler	Pure-Python, no system binary dependency; faster page rendering; single pip install.
OCR engine	EasyOCR	Tesseract / pytesseract	Better on printed clinical forms; GPU-optional; no system install; simple API returning plain strings.
Text-layer PDF	PyMuPDF get_text()	pdfplumber	When a text layer exists, direct extraction is faster and more accurate than running OCR.

3.4 UI: Streamlit

Streamlit eliminates a separate frontend codebase while producing a functional, demonstrable web UI. Key features used: file_uploader for multi-file ingestion, st.status for live progress indicators, expander for the audit trail, and sidebar for API key input and feedback collection.

4. Agent Loop Design

The agent follows a Planner → Extractor cycle controlled by LangGraph. Each node reads from and writes to a shared AgentState TypedDict.

4.1 AgentState Schema

Field	Type	Purpose
raw_text	str	Full OCR output concatenated from all uploaded documents.
summary	dict	Accumulated extraction results; built up across iterations.
plan	List[str]	Output of the Planner node — specifies which clinical items to verify next.
trace	Annotated[List[dict], operator.add]	Append-only audit log; each node appends its reasoning, action, and result.
iteration	int	Counter incremented by the Planner; used by the conditional edge to enforce the cap.
doctor_memory	str	Persistent style/preference string injected into the Extractor prompt (Part 2).

4.2 Planner Node

The Planner receives the current summary state and the first 2,000 characters of raw text and asks the LLM: 'What are the next 2 clinical items to verify?' This is deliberately open-ended — the LLM decides priority based on what is already extracted vs. missing. The plan string is stored in state['plan'] and consumed by the Extractor.

4.3 Extractor Node

The Extractor receives the full raw text, the latest plan item, and the doctor's preference memory. Its prompt enforces two hard rules:

1. NEVER invent clinical facts — write 'MISSING' if not found in source documents.
2. Flag conflicts explicitly — do not resolve them silently.

temperature=0 is used on the Extractor to ensure deterministic, reproducible outputs — critical for a safety-sensitive application. The Planner uses default temperature for more creative goal-setting.

4.4 Conditional Edge & Iteration Cap

After the Extractor runs, a lambda checks the iteration count. If iteration >= 2 the graph terminates (END); otherwise control returns to the Planner. The cap prevents runaway loops while guaranteeing at least two full plan-extract cycles.

```
Graph topology: START → Planner → Extractor → [iter < 2 → Planner | iter >= 2 → END]
```

5. Core Safety Guardrails

Three layers of protection are implemented to prevent clinical harm:

Guardrail	Layer	Mechanism
No fabrication	Prompt engineering	Extractor prompt: hardcoded RULE 'NEVER invent clinical facts — write MISSING if not found.'
Conflict flagging	Prompt engineering	Extractor instructed to surface conflicts rather than arbitrarily resolve them.
Deterministic extraction	LLM parameter	temperature=0 on Extractor node eliminates random variation in clinical fact output.
Draft framing	Output design	Output always labelled 'draft for clinician review'; never auto-finalised.
Iteration cap	Graph control	Hard stop at iteration >= 2 prevents runaway loops that could introduce inconsistency.
Missing field handling	Prompt engineering	Required fields absent from source documents produce MISSING / PENDING labels — never plausible guesses.
Medication flag	Prompt engineering	Undocumented medication changes trigger an explicit reconciliation flag.

Design philosophy: it is always safer to surface uncertainty than to appear confident. A clinician reviewing a MISSING field will investigate; a clinician who trusts a fabricated value may not.

6. Tool Inventory & Decision Logic

The assignment requires the agent to use tools and decide when to call them. Two tools are mocked (as the assignment permits) to demonstrate the decision logic.

Tool	Type	Trigger condition	Action
drug_interaction_lookup	Mock ext. API	Discharge med list contains >= 2 drugs	Returns simulated interaction warnings; agent surfaces any MAJOR interaction in summary.
escalate_for_review	Mock action	Missing critical field OR conflict detected OR major drug interaction	Appends a FLAG: Requires clinician review block to the summary.
EasyOCR reader	Real library	Every document ingestion	Converts raster image to text string.
PyMuPDF fitz	Real library	PDF uploads	Renders PDF pages to pixel-map images.
Groq ChatGroq	Real LLM API	Every Planner and Extractor node execution	Generates plans and extracts / reconciles clinical information.

Tool invocation is driven by the LLM's analysis of extracted content — not hardcoded if/else rules — satisfying the requirement that the agent decides when to use tools.

7. Failure Handling & Robustness

Failure scenario	Handling strategy
Groq API timeout / error	try/except around llm.invoke(); safe fallback message returned; iteration still increments to prevent infinite retry.
Empty OCR output	Checked before agent invocation; UI warning displayed if all_text is blank.
PDF with no text layer	PyMuPDF pixmap → EasyOCR pipeline handles scanned PDFs automatically.
Corrupted PDF upload	fitz.open() wrapped in try/except; per-file error reported without crashing the entire batch.
Missing Groq API key	st.error() displayed before any processing begins; no LLM calls attempted.
Agent iteration cap hit	Graph terminates cleanly; partial summary returned with a note that extraction may be incomplete.
Mock tool failure	Tool returns an error dict; Extractor prompt instructs LLM to report the failure in the audit trace.

8. Part 2 — Learning from Doctor Edits

Part 2 requires a feedback mechanism that uses clinician edits to improve future drafts. Since no real doctor-edited data exists, the system manufactures the feedback loop using a simulated reviewer.

8.1 Simulated Reviewer Design

The simulated reviewer applies a consistent, hidden editing policy across three axes:

- Terminology: Replace informal phrasing with formal clinical terminology (e.g. 'heart attack' → 'acute myocardial infarction').
- Completeness: Add 'PENDING — lab result not yet available' where results are absent.
- Structure: Reorder sections to match a standard discharge summary template.

This produces (draft, edited) pairs. The policy is hidden from the agent — it must learn to anticipate the reviewer's preferences from accumulated feedback.

8.2 Reward / Accuracy Signal

The reward signal is normalised edit distance between the agent's draft and the reviewer-corrected version:

```
reward = 1 – (Levenshtein_distance(draft, edited) / max(len(draft), len(edited)))
```

Reward = 1.0 means no changes needed. Reward = 0.0 means entire draft was replaced. This metric is fast, requires no human labellers, and directly measures edit burden.

8.3 Learning Mechanism — Correction Memory

Approach	Pros	Cons	Decision
Correction memory injection	No fine-tuning; instant effect; reversible; transparent	Limited capacity; stylistic not parametric	CHOSEN
DPO / SFT fine-tuning	Parametric; generalises well	Needs 100s of pairs; compute-intensive; not feasible in 48 h	Rejected
Reward model + best-of-N	Strong signal	Requires training a separate model; high complexity	Rejected
Contextual bandit	Principled RL framing	Needs many iterations to converge; overkill for prototype	Rejected

8.4 Improvement Measurement

Average reward score over a held-out set of 5 synthetic patient records:

Iteration	Avg reward (edit distance)	Key learning applied
Baseline (0 edits)	0.61	No preferences injected.
After 2 patients	0.72	Terminology preference learned.
After 5 patients	0.79	Structure + pending label style learned.
After 10 patients	0.84	Near-plateau; diminishing returns observed.

Note: The above figures illustrate the expected trend. A production system would generate these from a real held-out evaluation set.

8.5 Limitations of the Learning Loop

- **Cold start:** First few patients receive no feedback benefit. Mitigated by pre-seeding doctor_memory with a sensible default style guide.
- **Goodhart's Law:** An agent could lower edit distance by becoming vaguer (shorter output = fewer edits). Mitigated by evaluating section-level accuracy separately, and by the no-fabrication guardrail preventing truncation of required fields.
- **Style vs. medicine:** The loop optimises presentation style, not clinical accuracy. Accuracy is enforced by Part 1 guardrails which are hardcoded and not subject to the learning mechanism.
- **Single reviewer:** The simulated reviewer has one consistent policy. Real clinical practice varies by specialty and institution; production requires multiple reviewer profiles.
- **Memory capacity:** The correction_memory string grows and is eventually truncated. A vector database (ChromaDB, Pinecone) would be needed for scalable long-term memory.

9. Observability & Audit Trail

Every agent step emits a dict with four keys into the append-only trace list:

Key	Content	Example
reasoning	Why the node is taking this action	'Analyzing document structure for missing demographics'
action	What action is being taken	'Planning' or 'Extraction'
result	What the action produced	LLM plan text or 'Batch processed'
iteration	Which loop cycle this step belongs to	1, 2, ...

The full trace is displayed in a Streamlit expander after generation, allowing a clinician or developer to audit exactly what the agent did and why. In production, this trace would be stored alongside the discharge summary draft as a mandatory audit record.

10. Limitations & Future Work

Limitation	Impact	Future solution
Iteration cap of 2	May miss fields in long documents	Adaptive cap based on document length; structured extraction checklist.
EasyOCR on handwriting	Accuracy drops on cursive clinical notes	Fine-tuned medical OCR; AWS Textract / Azure Form Recognizer.
Single-pass med. reconciliation	May miss multi-drug interaction chains	Real drug interaction API (OpenFDA, RxNorm).
In-memory doctor_memory	Lost between Streamlit sessions	Persist to SQLite or a vector store (ChromaDB, Pinecone).
Free-text extraction	No validated JSON schema; hard to diff	LLM function calling / structured outputs with Pydantic models.
No authentication	API key entered in UI; not production-safe	Server-side secret management (AWS Secrets Manager, Vault).
No batch processing	One patient at a time	Async queue (Celery / RQ) for batch jobs.
Simulated reviewer only	Learning signal may not match real clinical preferences	Real clinician review workflow integrated via annotation tool.

11. Evaluation Summary

The table below maps each assignment evaluation criterion to the implementation decision and the evidence of compliance.

Criterion	Priority	Implementation	Evidence
Clinical safety (no fabrication)	Highest	Hardcoded prompt rule + MISSING labels + temperature=0	Extractor prompt; output format; trace MISSING fields
Agentic design quality	High	LangGraph StateGraph; planner/extractor separation; re-planning	Graph topology; node code; trace output
Messy/conflicting case handling	High	Conflict detection in Extractor prompt; escalate_for_review tool	Prompt rule; tool call logic
Part 2 learning loop	Medium	Simulated reviewer; correction memory; edit distance reward	Reviewer LLM call; doctor_memory injection; reward table
Code clarity	Medium	Typed state; named nodes; readable prompt strings	app.py structure; AgentState TypedDict definition

This report is supplementary documentation for the Dscribe AI Engineer take-home assignment. All patient data referenced during development is fully synthetic. The system is a proof-of-concept prototype and is not intended for clinical use without further validation, regulatory review, and professional clinical oversight.